



Universidad Nacional de San Luis
Facultad de Ciencias Físico Matemáticas y Naturales
Departamento de Informática

Práctico Final Integrador
Sistemas Distribuidos y Paralelos

Generación de Cartoons a partir de Imágenes

Juan Cruz Paez
María Lujan Flores Medina

Profesores:
Fabiana Piccoli
Cesar Ochoa

Ingeniería en Informática

2026

Índice general

1. Introducción	1
2. Decisiones de Diseño	3
2.1. Paradigmas y estrategia general	3
2.2. Algoritmo secuencial	3
2.3. Algoritmo con memoria compartida (OpenMP)	3
2.4. Algoritmo con memoria distribuida (MPI)	4
2.5. Algoritmo híbrido (MPI + OpenMP)	4
3. Estadísticas y Métricas de Rendimiento	5
3.1. Métricas de Rendimiento	5
3.2. Configuraciones Experimentales	6
3.3. Entornos de Ejecución	6
3.4. Visualizaciones de Desempeño	7
3.4.1. Speedup Promedio por Configuración	7
3.4.2. Eficiencia Paralela	7
3.4.3. Heatmap de Speedup por Escenario	8
3.4.4. Curva de Escalabilidad	9
4. Conclusiones y Análisis de Resultados	11
4.1. En el entorno local	11
4.2. En el entorno de el cluster	11
A. Resultados del Benchmark en Entorno Local (Compu Juan)	13
B. Resultados del Benchmark en Entorno Cluster	20

Capítulo 1

Introducción

El procesamiento digital de imágenes constituye una de las áreas más demandantes en términos computacionales dentro de la informática moderna. Si bien las bases matemáticas de muchas de sus técnicas datan de siglos atrás, su aplicación práctica a escala masiva solo fue posible con el avance tecnológico del hardware de las últimas décadas, que permitió realizar en tiempo razonable la enorme cantidad de operaciones aritméticas involucradas.

Una imagen digital puede representarse como un arreglo bidimensional de píxeles. En el modelo de color **RGB** (Red, Green, Blue) utilizado en este trabajo, cada píxel contiene tres valores enteros en el rango $[0, 255]$, uno por canal de color. Esta representación permite codificar hasta 16,8 millones de tonalidades distintas, superando la capacidad discriminatoria del ojo humano. Los casos extremos son el negro $(0, 0, 0)$ y el blanco $(255, 255, 255)$.

El objetivo del presente trabajo es la **generación automática de imágenes tipo *cartoon*** a partir de una imagen original en formato RGB. Esta transformación busca simplificar formas, resaltar contornos y reducir variaciones tonales, logrando una apariencia similar a la de una ilustración o dibujo animado. Para ello se aplican dos procesos independientes sobre la imagen original:

- **Posterización:** reduce el rango total de niveles de color a un conjunto más pequeño de valores discretos. Cada píxel pasa a adoptar el valor del nivel más cercano dentro de ese rango reducido. En imágenes a color se posteriza cada canal por separado y luego se suman las imágenes resultantes.
- **Detección de bordes (borroneado):** pipeline de tres etapas secuencialmente dependientes entre sí:
 1. *Filtrado (blur)*: suaviza la imagen para eliminar ruido.
 2. *Resaltado (highlight)*: amplifica las diferencias entre zonas de borde y no-borde mediante aproximación de gradiente (filtros Prewitt, Sobel u otros).
 3. *Umbralado*: binariza el resultado del resaltado para obtener los bordes detectados.

La imagen cartoon final se obtiene sumando la salida de la posterización y la de la detección de bordes.

Dado el volumen de cómputo involucrado, este trabajo explora y compara distintas **estrategias de paralelización** con el fin de reducir el tiempo de procesamiento. Se implementan cuatro versiones del algoritmo:

- **Secuencial:** implementación de referencia, sin paralelismo.
- **Memoria compartida (OpenMP):** paralelismo a nivel de threads dentro de un único nodo mediante directivas.
- **Memoria distribuida (MPI):** paralelismo mediante pasaje de mensajes entre procesos.
- **Híbrido (MPI + OpenMP):** combinación de ambos paradigmas para explotar simultáneamente múltiples nodos y múltiples núcleos por nodo.

El análisis de desempeño se realiza sobre imágenes de tres tamaños (800×800 , 2000×2000 y 5000×5000 píxeles), con filtros de convolución de 3×3 y 5×5 , y posterizado configurado con 3 y 9 rangos de color. Las métricas utilizadas son el *speedup* y la eficiencia paralela.

Capítulo 2

Decisiones de Diseño

Paradigmas y estrategia general

Para el diseño de los algoritmos paralelos se adoptaron dos paradigmas de forma complementaria:

- **Paradigma de control** para las tareas de posterizado y borroneado: dado que ambas operan de forma independiente sobre la imagen de entrada (sin dependencias de datos entre sí), pueden ejecutarse en paralelo. Se decidió paralelizarlas asignando recursos de cómputo de forma independiente a cada una.
- **Paradigma de datos** para la distribución de los datos de entrada: dentro de cada tarea, los datos (filas o bloques de la imagen) se reparten entre los workers disponibles.

Dentro de la sección de borroneado, las dos subtareas (blur y highlight+umbralado) sí presentan dependencia secuencial entre sí, por lo que se implementa un **pipeline de datos**: primero se ejecuta la función de blur sobre toda la imagen y, sobre el resultado, se aplica la función de highlight junto con la operación de umbralado.

Algoritmo secuencial

En todas las versiones del algoritmo se aplicó la técnica de **padding**: se añade un borde auxiliar de píxeles con valor neutro (negro) alrededor de la matriz de entrada. Esto elimina la necesidad de verificar en cada iteración si las celdas vecinas son válidas al calcular el blur y el highlight, simplificando el código y mejorando la localidad de memoria.

Algoritmo con memoria compartida (OpenMP)

Se aplica descomposición de datos de entrada para cada tarea dentro del proceso.

La paralelización de las tareas de borroneado y posterizado se realiza con la directiva

```
#pragma omp parallel sections num_threads(2)
```

que identifica las dos secciones a ejecutar en paralelo, combinada con `#pragma omp section` para cada una de ellas.

Suma y posterizado. Se usa `#pragma omp parallel for schedule(static)`. La planificación estática se eligió porque el costo computacional de cada iteración es uniforme, lo que permite dividir el espacio de trabajo de forma equitativa sin sobrecarga de administración dinámica.

Blur y highlight. La inicialización de datos se paraleliza con `#pragma omp parallel for schedule(static)`. El procesamiento principal emplea

```
#pragma omp parallel for collapse(2) schedule(guided)
```

La cláusula `collapse(2)` fusiona los dos loops anidados en un único espacio de iteraciones. La planificación `guided` se adopta porque el trabajo por iteración puede ser variable o impredecible, y resulta más eficiente que `dynamic` en la administración del balance de carga.

Algoritmo con memoria distribuida (MPI)

El proceso de rango 0 actúa como **orquestador**: recibe la imagen original, la distribuye entre los workers y recolecta los resultados parciales. Los procesos worker se dividen en dos grupos según la sección asignada: la mitad procesa el posterizado y la otra mitad el borronado.

Durante el análisis experimental se observó que paralelizar la **suma final** de las matrices de posterizado y borronado incrementaba el tiempo de ejecución de forma contraproducente. Este comportamiento se atribuye a la latencia y overhead de comunicación entre procesos, que supera el costo de la suma en sí misma. Por esta razón se decidió **no paralelizar la suma final**, dejándola a cargo del proceso orquestador.

Algoritmo híbrido (MPI + OpenMP)

El algoritmo híbrido fue creado:

- MPI: Se reutilizó el código generado en el algoritmo con memoria distribuida.
- OpenMP: Se agregaron a los bucles del anterior código, las directrices que se utilizaron previamente en el algoritmo con memoria compartida.

La suma que se realiza al final para juntar los resultados del posterizado y borronado se decidió que se paralelizara utilizando únicamente OpenMP, como se vio en el anterior algoritmo, los tiempos aumentaban al paralelizarla con MPI pero OpenMP al menos en un entorno local resulta ser mucho más eficiente.

Capítulo 3

Estadísticas y Métricas de Rendimiento

Se detallan las métricas y estadísticas de rendimiento recopiladas durante las ejecuciones de prueba del generador de imágenes tipo *cartoon*. Se describen los entornos de prueba, las configuraciones experimentales analizadas y se presentan los gráficos resultantes para visualizar el comportamiento de las distintas implementaciones paralelas.

Métricas de Rendimiento

Para evaluar y comparar cuantitativamente el desempeño de las implementaciones paralelas en relación con la versión secuencial de referencia, se utilizaron las siguientes métricas estándar:

- **Tiempo de ejecución (T_p):** El tiempo total medido en segundos empleado en el procesamiento de la imagen. Para garantizar la precisión de la medición del algoritmo en sí, este tiempo excluye las operaciones de entrada/salida (I/O) en disco (carga de la imagen original y guardado del archivo final procesado).
- **Speedup o Aceleración (S_p):** Representa la ganancia de velocidad obtenida al utilizar recursos paralelos en comparación con el algoritmo secuencial. Se define mediante la fórmula:

$$S_p = \frac{T_1}{T_p}$$

Donde T_1 es el tiempo de ejecución secuencial y T_p es el tiempo de ejecución con la versión paralela utilizando p unidades de recursos (threads o procesos). Un speedup ideal sería uno super lineal.

- **Eficiencia Paralela (E_p):** Mide la fracción de tiempo durante la cual los procesadores asignados son utilizados efectivamente para realizar cálculos útiles. Se define como:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p}$$

Una eficiencia de 1,0 (100 %) indica que todos los recursos asignados están trabajando a su máxima capacidad sin pérdidas por sobrecarga de sincronización, desbalanceo de carga o comunicación.

Configuraciones Experimentales

Las pruebas de rendimiento se llevaron a cabo variando múltiples parámetros del algoritmo y del entorno para analizar el comportamiento bajo distintas intensidades de cómputo y comunicación:

1. **Resolución de las Imágenes:** Se utilizaron tres resoluciones con distinta cantidad de carga de trabajo:
 - **Baja** (800×800 píxeles): Carga de trabajo ligera (640 mil píxeles).
 - **Media** (2000×2000 píxeles): Carga de trabajo moderada (4 millones de píxeles).
 - **Alta** (5000×5000 píxeles): Carga de trabajo pesada (25 millones de píxeles).
2. **Intensidad del Filtro de Convolución (Borroneado):**
 - **Filtro 3×3 (Radio 1):** Requiere menor cantidad de operaciones por píxel.
 - **Filtro 5×5 (Radio 2):** Requiere mayor cantidad de operaciones de vecindad por píxel.
3. **Rangos de Posterizado:**
 - **3 Rangos (Parámetro 1):** Pocos intervalos de discretización de color.
 - **9 Rangos (Parámetro 2):** Mayor granularidad en el cálculo del color final.
4. **Recursos y Paradigmas Asignados:**
 - **Secuencial:** Ejecución base con un solo thread/proceso (1 recurso).
 - **OpenMP (Memoria Compartida):** Evaluado con 2, 4, 8, 16 y 32 threads en el cluster (y hasta 8 threads en local).
 - **MPI (Memoria Distribuida):** Evaluado con 3, 8, 16 y 32 procesos en el cluster (y hasta 8 procesos en local).
 - **Híbrido (MPI + OpenMP):** Evaluado en combinaciones de procesos $\times$ threads: 3×2 , 3×4 , 3×8 , 4×8 , 8×4 y 16×2 en el cluster (y hasta 5×2 en local).

Entornos de Ejecución

Para contrastar el comportamiento del algoritmo bajo diferentes arquitecturas físicas, las pruebas se corrieron en dos entornos:

- **Entorno Local (Prefijo CJ):** Ejecución sobre una máquina personal multiprocesador.
- **Entorno de cluster (Prefijo RC):** Ejecución sobre el cluster del departamento de informática, que permite evaluar el rendimiento del paso de mensajes entre múltiples nodos de red físicos y la escalabilidad real en sistemas distribuidos.

Visualizaciones de Desempeño

A continuación, se presentan los gráficos generados para representar de manera sintética el comportamiento del sistema.

3.4.1. Speedup Promedio por Configuración

Las siguientes figuras muestran el Speedup promedio (calculado sobre todos los filtros y posterizados) clasificado por el tamaño de la imagen. Las configuraciones están ordenadas numéricamente para apreciar la progresión lógica de los recursos asignados.

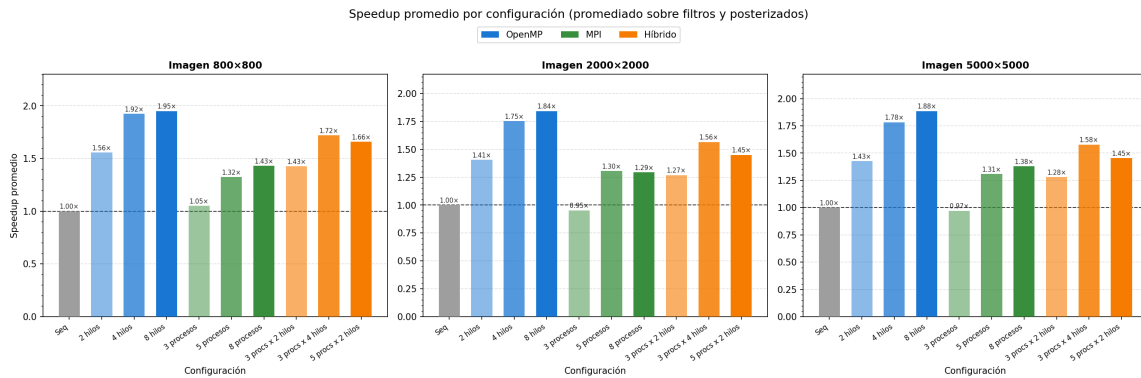


Figura 3.1: Speedup promedio por configuración en Entorno Local para imágenes de 800×800 , 2000×2000 y 5000×5000 .

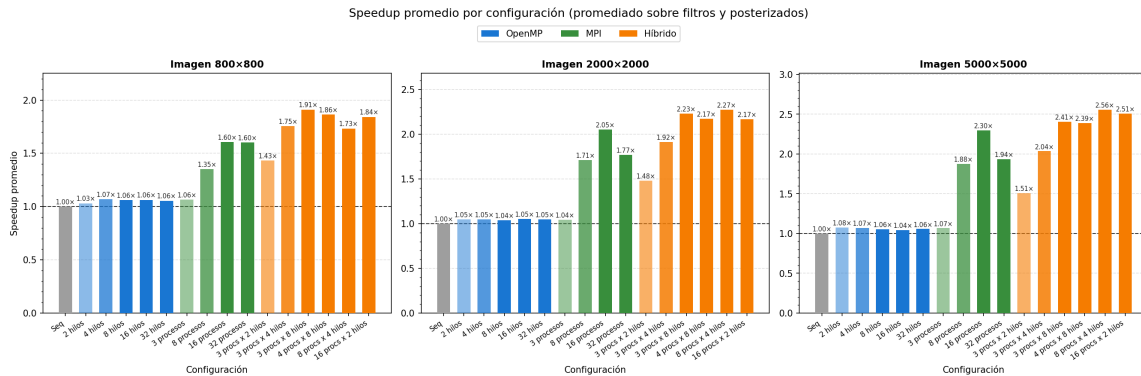


Figura 3.2: Speedup promedio por configuración en Entorno cluster para imágenes de 800×800 , 2000×2000 y 5000×5000 .

3.4.2. Eficiencia Paralela

La eficiencia representa qué tan efectivamente se están usando los recursos agregados. El valor ideal es 1,0.

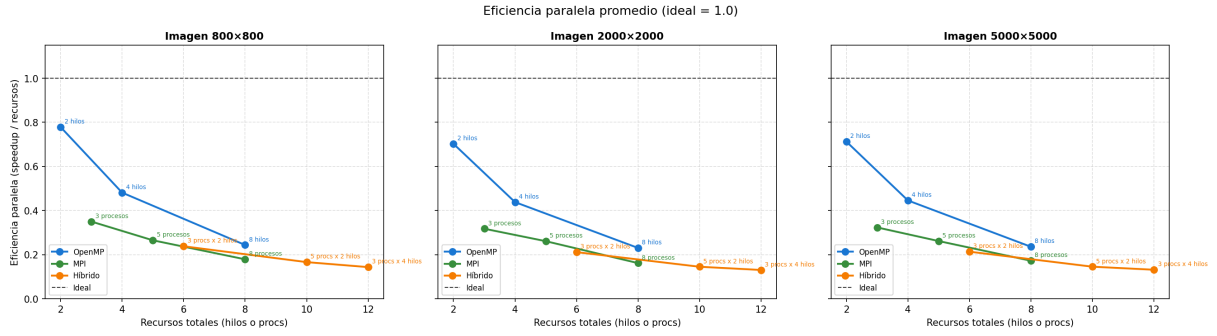


Figura 3.3: Eficiencia paralela promedio en entorno local comparando OpenMP, MPI e Híbrido frente al comportamiento Ideal.

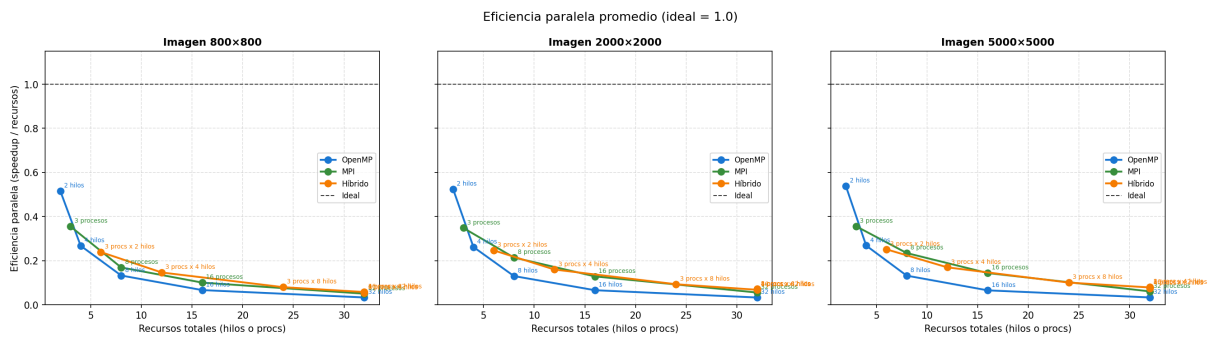


Figura 3.4: Eficiencia paralela promedio en Entorno cluster comparando OpenMP, MPI e Híbrido frente al comportamiento Ideal.

3.4.3. Heatmap de Speedup por Escenario

El heatmap permite cruzar todas las ejecuciones (combinaciones de resolución, filtro y posterización en el eje X) con todas las configuraciones de recursos (eje Y), mapeando el speedup absoluto en una escala de color.

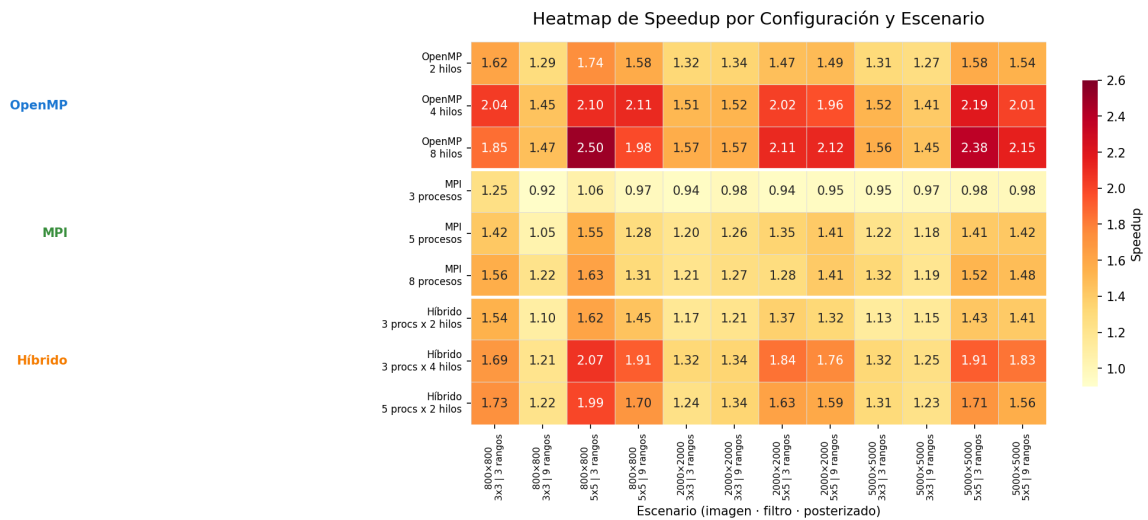


Figura 3.5: Heatmap de Speedup detallado por escenario y configuración en entorno local.

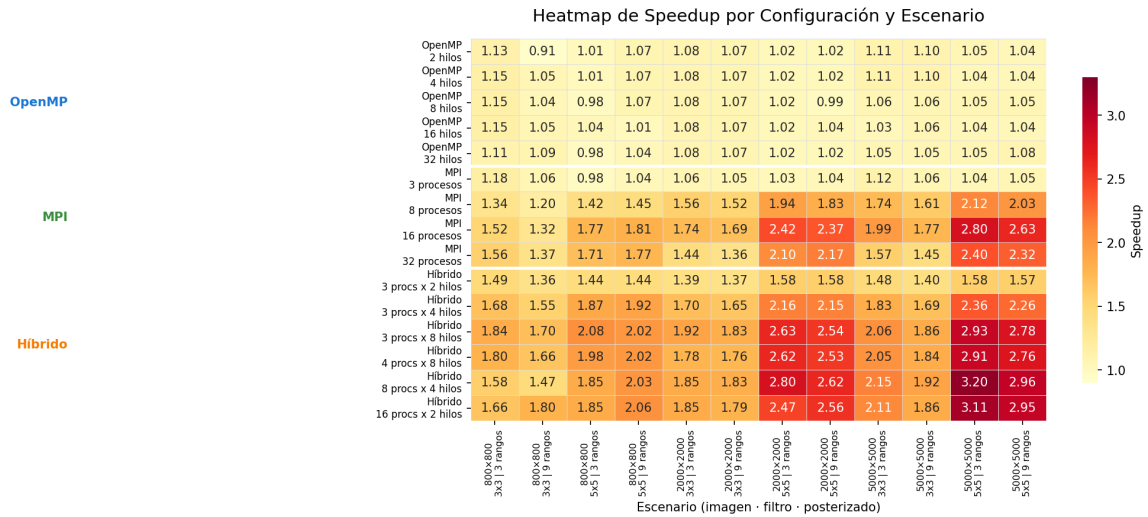


Figura 3.6: Heatmap de Speedup detallado por escenario y configuración en el cluster.

3.4.4. Curva de Escalabilidad

Muestra la evolución del speedup de la mejor configuración de cada paradigma a medida que el problema crece en volumen (de 800×800 a 5000×5000 píxeles).

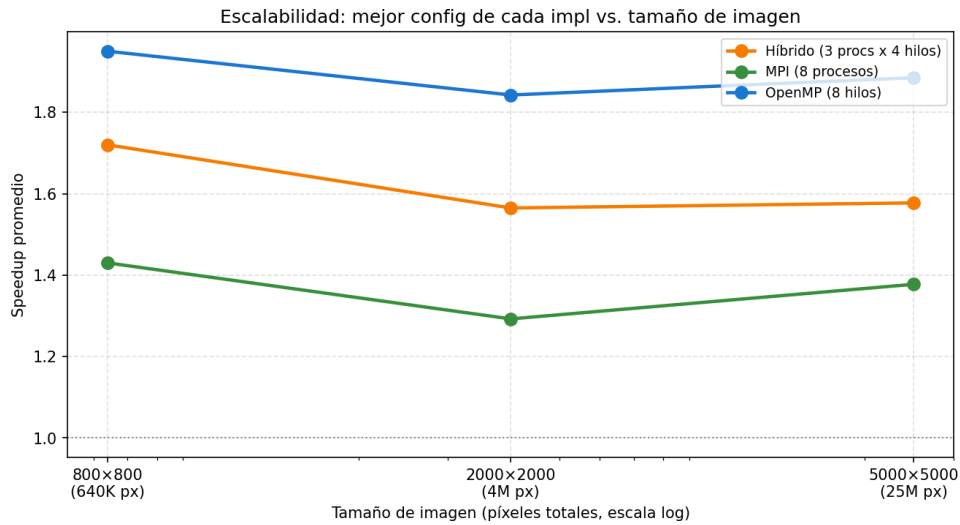


Figura 3.7: Escalabilidad: Speedup promedio de la mejor configuración de cada implementación en función del tamaño de la imagen (local).

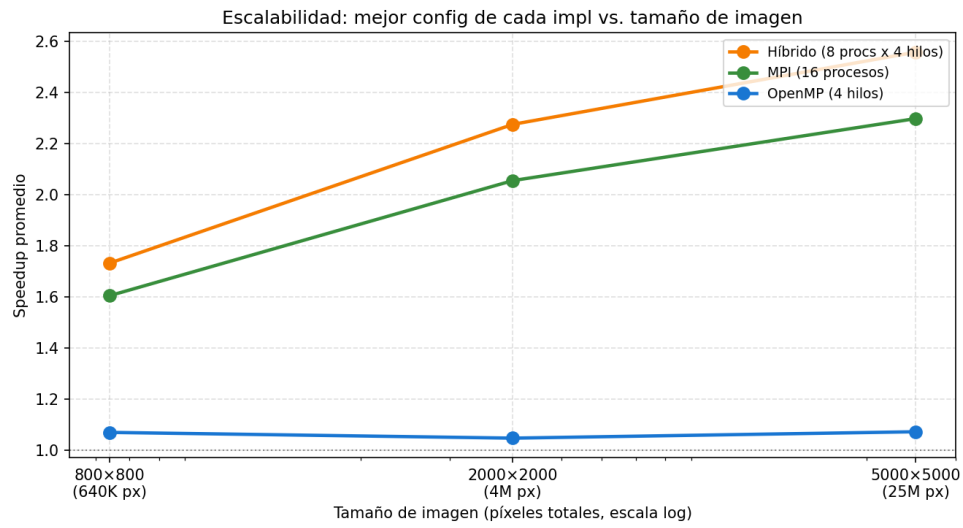


Figura 3.8: Escalabilidad: Speedup promedio de la mejor configuración de cada implementación en función del tamaño de la imagen (cluster).

Nota: Las tablas numéricas de datos crudos obtenidas de cada ejecución se encuentran adjuntas en los Anexos A y B al final de este informe.

Capítulo 4

Conclusiones y Análisis de Resultados

Se presenta el análisis de los resultados experimentales obtenidos a partir de las pruebas de rendimiento del generador de *cartoons*. Se contrastan las diferentes implementaciones y se analizan las causas arquitectónicas, de software y de entorno físico que justifican el comportamiento observado en los gráficos.

En el entorno local

El comportamiento de memoria compartida (OpenMP) en el entorno local es el que muestra una gran mejora en los tiempos con respecto a los demás algoritmos. Se asume esto debido a como no se duplican matrices ni se hacen envíos de mensajes, los threads acceden directamente a la imagen cargada en la RAM. De esta forma se obtiene la gran ventaja de OpenMP sobre MPI en todas las configuraciones y resoluciones.

- El techo de rendimiento se encuentra en este caso en los 4 threads, y cuando se hace el filtro 5×5 , creemos que esto es así por que se tiene el triple de multiplicaciones por pixel. Entonces el speedup mejora porque la carga computacional justifica dividir el trabajo en threads

En los algoritmos de MPI y el híbrido consideramos que si bien los resultados mejoran los tiempos del secuencial, estas mejoras son menores que en OpenMP por el tiempo de overhead que se tiene por el pasaje de mensajes. Esto es tiempo en el que OpenMP gana considerable ventaja.

En el entorno de el cluster

El comportamiento de la implementación de memoria compartida (OpenMP) en el cluster muestra que el speedup permanece prácticamente plano (oscilando entre $0,91 \times$ y $1,15 \times$) sin importar que la cantidad de threads aumente de 2 a 32. Se asume que tal comportamiento se atribuye a dos factores: En los entornos de cluster (como SLURM), cuando se ejecuta un proceso multiprocesador de forma nativa sin configurar directivas de asignación específicas (por ejemplo, omitiendo la reserva de múltiples núcleos virtuales por tarea), el sistema operativo limita por afinidad (*affinity binding*) el proceso completo

a un único núcleo físico de CPU. Como consecuencia, aunque la directiva OpenMP en software cree 4, 8, 16 o 32 threads, todos estos threads son forzados por el kernel del sistema operativo a alternarse en el mismo procesador. Esto no solo impide la computación paralela real, sino que añade sobrecarga debido al intercambio de contexto (*context switching*) y a la invalidación de caché de datos.

El diseño del algoritmo divide la carga mediante `#pragma omp parallel sections` asignando un thread al posterizado y otro al borronado. Dado que la convolución bidimensional del borronado (filtro de blur y Sobel) requiere miles de operaciones aritméticas en punto flotante por píxel, esta sección tarda considerablemente más tiempo que el posterizado (que consiste en una división simple por píxel). Al ser la sección de borronado el cuello de botella masivo, y al ejecutarse esta de forma secuencial debido a la restricción de afinidad sobre un solo procesador, el speedup general queda limitado por la Ley de Amdahl a un máximo de $\approx 1,15\times$, haciendo que la adición de threads no tenga impacto real.

A diferencia de OpenMP, las implementaciones que utilizan pasaje de mensajes (MPI) y la version híbrida logran mostrar una *escalabilidad positiva*: Al invocar el programa mediante `mpirun -n p`, se crean p procesos independientes del sistema operativo; así, el planificador del sistema y el entorno MPI distribuyen estos procesos en diferentes núcleos físicos. En cuanto a speedup se observa que tanto para MPI como para la configuración híbrida, el mismo aumenta conforme la imagen es más grande.

Apéndice A

Resultados del Benchmark en Entorno Local (Compu Juan)

En este anexo se presentan las tablas de tiempos, speedup y eficiencia obtenidos en el entorno local.

Imagen: 800x800

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.041045	1.00x	1.00
OpenMP	2 hilos	0.031338	1.31x	0.65
OpenMP	4 hilos	0.025477	1.61x	0.40
OpenMP	8 hilos	0.025471	1.61x	0.20
MPI	3 procesos	0.042386	0.97x	0.32
MPI	5 procesos	0.03515	1.17x	0.23
MPI	8 procesos	0.03181	1.29x	0.16
Híbrido	3 procs x 2 hilos	0.034013	1.21x	0.20
Híbrido	3 procs x 4 hilos	0.029912	1.37x	0.11
Híbrido	5 procs x 2 hilos	0.030646	1.34x	0.13

Figura A.1: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.045989	1.00x	1.00
OpenMP	2 hilos	0.035314	1.30x	0.65
OpenMP	4 hilos	0.030444	1.51x	0.38
OpenMP	8 hilos	0.031772	1.45x	0.18
MPI	3 procesos	0.048763	0.94x	0.31
MPI	5 procesos	0.042867	1.07x	0.21
MPI	8 procesos	0.037617	1.22x	0.15
Híbrido	3 procs x 2 hilos	0.038705	1.19x	0.20
Híbrido	3 procs x 4 hilos	0.035905	1.28x	0.11
Híbrido	5 procs x 2 hilos	0.03321	1.38x	0.14

Figura A.2: Resultados del benchmark en entorno local para imagen de 800x800, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.071618	1.00x	1.00
OpenMP	2 hilos	0.044335	1.62x	0.81
OpenMP	4 hilos	0.032426	2.21x	0.55
OpenMP	8 hilos	0.031636	2.26x	0.28
MPI	3 procesos	0.075462	0.95x	0.32
MPI	5 procesos	0.047112	1.52x	0.30
MPI	8 procesos	0.046491	1.54x	0.19
Híbrido	3 procs x 2 hilos	0.048047	1.49x	0.25
Híbrido	3 procs x 4 hilos	0.036771	1.95x	0.16
Híbrido	5 procs x 2 hilos	0.038681	1.85x	0.19

Figura A.3: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.072705	1.00x	1.00
OpenMP	2 hilos	0.046921	1.55x	0.77
OpenMP	4 hilos	0.035794	2.03x	0.51
OpenMP	8 hilos	0.033971	2.14x	0.27
MPI	3 procesos	0.07505	0.97x	0.32
MPI	5 procesos	0.048773	1.49x	0.30
MPI	8 procesos	0.049505	1.47x	0.18
Híbrido	3 procs x 2 hilos	0.05103	1.42x	0.24
Híbrido	3 procs x 4 hilos	0.044933	1.62x	0.13
Híbrido	5 procs x 2 hilos	0.04826	1.51x	0.15

Figura A.4: Resultados del benchmark en entorno local para imagen de 800x800, filtro 5x5 y 9 rangos de posterización.

Imagen: 2000x2000**Filtro: 3x3, Posterizado: 3 rangos**

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.273986	1.00x	1.00
OpenMP	2 hilos	0.203147	1.35x	0.67
OpenMP	4 hilos	0.182397	1.50x	0.38
OpenMP	8 hilos	0.182454	1.50x	0.19
MPI	3 procesos	0.278488	0.98x	0.33
MPI	5 procesos	0.221218	1.24x	0.25
MPI	8 procesos	0.217963	1.26x	0.16
Híbrido	3 procs x 2 hilos	0.224183	1.22x	0.20
Híbrido	3 procs x 4 hilos	0.191898	1.43x	0.12
Híbrido	5 procs x 2 hilos	0.193905	1.41x	0.14

Figura A.5: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.296771	1.00x	1.00
OpenMP	2 hilos	0.235115	1.26x	0.63
OpenMP	4 hilos	0.208753	1.42x	0.36
OpenMP	8 hilos	0.20036	1.48x	0.19
MPI	3 procesos	0.29319	1.01x	0.34
MPI	5 procesos	0.240458	1.23x	0.25
MPI	8 procesos	0.23891	1.24x	0.16
Híbrido	3 procs x 2 hilos	0.239332	1.24x	0.21
Híbrido	3 procs x 4 hilos	0.216058	1.37x	0.11
Híbrido	5 procs x 2 hilos	0.234226	1.27x	0.13

Figura A.6: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.511899	1.00x	1.00
OpenMP	2 hilos	0.320325	1.60x	0.80
OpenMP	4 hilos	0.241046	2.12x	0.53
OpenMP	8 hilos	0.223295	2.29x	0.29
MPI	3 procesos	0.466783	1.10x	0.37
MPI	5 procesos	0.321924	1.59x	0.32
MPI	8 procesos	0.363982	1.41x	0.18
Híbrido	3 procs x 2 hilos	0.317579	1.61x	0.27
Híbrido	3 procs x 4 hilos	0.244634	2.09x	0.17
Híbrido	5 procs x 2 hilos	0.278002	1.84x	0.18

Figura A.7: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.481179	1.00x	1.00
OpenMP	2 hilos	0.318306	1.51x	0.76
OpenMP	4 hilos	0.236905	2.03x	0.51
OpenMP	8 hilos	0.227491	2.12x	0.26
MPI	3 procesos	0.480026	1.00x	0.33
MPI	5 procesos	0.326558	1.47x	0.29
MPI	8 procesos	0.37323	1.29x	0.16
Híbrido	3 procs x 2 hilos	0.323675	1.49x	0.25
Híbrido	3 procs x 4 hilos	0.247906	1.94x	0.16
Híbrido	5 procs x 2 hilos	0.285562	1.69x	0.17

Figura A.8: Resultados del benchmark en entorno local para imagen de 2000x2000, filtro 5x5 y 9 rangos de posterización.

Imagen: 5000x5000**Filtro: 3x3, Posterizado: 3 rangos**

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	1.717309	1.00x	1.00
OpenMP	2 hilos	1.301743	1.32x	0.66
OpenMP	4 hilos	1.140599	1.51x	0.38
OpenMP	8 hilos	1.189528	1.44x	0.18
MPI	3 procesos	1.749847	0.98x	0.33
MPI	5 procesos	1.338978	1.28x	0.26
MPI	8 procesos	1.217067	1.41x	0.18
Híbrido	3 procs x 2 hilos	1.389992	1.24x	0.21
Híbrido	3 procs x 4 hilos	1.250254	1.37x	0.11
Híbrido	5 procs x 2 hilos	1.262915	1.36x	0.14

Figura A.9: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	2.05788	1.00x	1.00
OpenMP	2 hilos	1.64827	1.25x	0.62
OpenMP	4 hilos	1.473558	1.40x	0.35
OpenMP	8 hilos	1.443685	1.43x	0.18
MPI	3 procesos	2.053074	1.00x	0.33
MPI	5 procesos	1.688571	1.22x	0.24
MPI	8 procesos	1.674865	1.23x	0.15
Híbrido	3 procs x 2 hilos	1.714778	1.20x	0.20
Híbrido	3 procs x 4 hilos	1.553586	1.32x	0.11
Híbrido	5 procs x 2 hilos	1.609253	1.28x	0.13

Figura A.10: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	2.878141	1.00x	1.00
OpenMP	2 hilos	1.821616	1.58x	0.79
OpenMP	4 hilos	1.34705	2.14x	0.53
OpenMP	8 hilos	1.265217	2.27x	0.28
MPI	3 procesos	2.865355	1.00x	0.33
MPI	5 procesos	1.89299	1.52x	0.30
MPI	8 procesos	1.560149	1.84x	0.23
Híbrido	3 procs x 2 hilos	1.889409	1.52x	0.25
Híbrido	3 procs x 4 hilos	1.412865	2.04x	0.17
Híbrido	5 procs x 2 hilos	1.66243	1.73x	0.17

Figura A.11: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	3.064857	1.00x	1.00
OpenMP	2 hilos	1.994967	1.54x	0.77
OpenMP	4 hilos	1.493682	2.05x	0.51
OpenMP	8 hilos	1.411506	2.17x	0.27
MPI	3 procesos	3.006962	1.02x	0.34
MPI	5 procesos	2.074393	1.48x	0.30
MPI	8 procesos	2.319496	1.32x	0.17
Híbrido	3 procs x 2 hilos	2.057191	1.49x	0.25
Híbrido	3 procs x 4 hilos	1.589104	1.93x	0.16
Híbrido	5 procs x 2 hilos	1.816961	1.69x	0.17

Figura A.12: Resultados del benchmark en entorno local para imagen de 5000x5000, filtro 5x5 y 9 rangos de posterización.

Apéndice B

Resultados del Benchmark en Entorno Cluster

En este anexo se presentan las tablas de tiempos, speedup y eficiencia obtenidos en el cluster del Departamento.

Imagen: 800x800

Filtro: 3x3, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.301987	1.00x	1.00
OpenMP	2 hilos	0.267195	1.13x	0.57
OpenMP	4 hilos	0.261739	1.15x	0.29
OpenMP	8 hilos	0.261934	1.15x	0.14
OpenMP	16 hilos	0.262042	1.15x	0.07
OpenMP	32 hilos	0.272949	1.11x	0.03
MPI	3 procesos	0.25624	1.18x	0.39
MPI	8 procesos	0.224576	1.34x	0.17
MPI	16 procesos	0.198748	1.52x	0.09
MPI	32 procesos	0.193907	1.56x	0.05
Híbrido	3 procs x 2 hilos	0.203023	1.49x	0.25
Híbrido	3 procs x 4 hilos	0.179971	1.68x	0.14
Híbrido	3 procs x 8 hilos	0.164229	1.84x	0.08
Híbrido	4 procs x 8 hilos	0.167661	1.80x	0.06
Híbrido	8 procs x 4 hilos	0.19117	1.58x	0.05
Híbrido	16 procs x 2 hilos	0.181443	1.66x	0.05

Figura B.1: Resultados del benchmark en entorno cluster para imagen de 800x800, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.297789	1.00x	1.00
OpenMP	2 hilos	0.328266	0.91x	0.45
OpenMP	4 hilos	0.283672	1.05x	0.26
OpenMP	8 hilos	0.285086	1.04x	0.13
OpenMP	16 hilos	0.283647	1.05x	0.07
OpenMP	32 hilos	0.272317	1.09x	0.03
MPI	3 procesos	0.281498	1.06x	0.35
MPI	8 procesos	0.247162	1.20x	0.15
MPI	16 procesos	0.225037	1.32x	0.08
MPI	32 procesos	0.217489	1.37x	0.04
Híbrido	3 procs x 2 hilos	0.219274	1.36x	0.23
Híbrido	3 procs x 4 hilos	0.191644	1.55x	0.13
Híbrido	3 procs x 8 hilos	0.175611	1.70x	0.07
Híbrido	4 procs x 8 hilos	0.179215	1.66x	0.05
Híbrido	8 procs x 4 hilos	0.202462	1.47x	0.05
Híbrido	16 procs x 2 hilos	0.165188	1.80x	0.06

Figura B.2: Resultados del benchmark en entorno cluster para imagen de 800x800, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.397814	1.00x	1.00
OpenMP	2 hilos	0.394848	1.01x	0.50
OpenMP	4 hilos	0.394964	1.01x	0.25
OpenMP	8 hilos	0.406005	0.98x	0.12
OpenMP	16 hilos	0.383862	1.04x	0.06
OpenMP	32 hilos	0.405811	0.98x	0.03
MPI	3 procesos	0.404002	0.98x	0.33
MPI	8 procesos	0.280972	1.42x	0.18
MPI	16 procesos	0.224629	1.77x	0.11
MPI	32 procesos	0.232128	1.71x	0.05
Híbrido	3 procs x 2 hilos	0.276062	1.44x	0.24
Híbrido	3 procs x 4 hilos	0.213121	1.87x	0.16
Híbrido	3 procs x 8 hilos	0.191535	2.08x	0.09
Híbrido	4 procs x 8 hilos	0.201373	1.98x	0.06
Híbrido	8 procs x 4 hilos	0.2148	1.85x	0.06
Híbrido	16 procs x 2 hilos	0.215335	1.85x	0.06

Figura B.3: Resultados del benchmark en entorno cluster para imagen de 800x800, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	0.410248	1.00x	1.00
OpenMP	2 hilos	0.38424	1.07x	0.53
OpenMP	4 hilos	0.384144	1.07x	0.27
OpenMP	8 hilos	0.384267	1.07x	0.13
OpenMP	16 hilos	0.406055	1.01x	0.06
OpenMP	32 hilos	0.395164	1.04x	0.03
MPI	3 procesos	0.393659	1.04x	0.35
MPI	8 procesos	0.282207	1.45x	0.18
MPI	16 procesos	0.22702	1.81x	0.11
MPI	32 procesos	0.232201	1.77x	0.06
Híbrido	3 procs x 2 hilos	0.285681	1.44x	0.24
Híbrido	3 procs x 4 hilos	0.21365	1.92x	0.16
Híbrido	3 procs x 8 hilos	0.203314	2.02x	0.08
Híbrido	4 procs x 8 hilos	0.202746	2.02x	0.06
Híbrido	8 procs x 4 hilos	0.202464	2.03x	0.06
Híbrido	16 procs x 2 hilos	0.198703	2.06x	0.06

Figura B.4: Resultados del benchmark en entorno cluster para imagen de 800x800, filtro 5x5 y 9 rangos de posterización.

Imagen: 2000x2000**Filtro: 3x3, Posterizado: 3 rangos**

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	1.441434	1.00x	1.00
OpenMP	2 hilos	1.339598	1.08x	0.54
OpenMP	4 hilos	1.340394	1.08x	0.27
OpenMP	8 hilos	1.339804	1.08x	0.13
OpenMP	16 hilos	1.339719	1.08x	0.07
OpenMP	32 hilos	1.338476	1.08x	0.03
MPI	3 procesos	1.358742	1.06x	0.35
MPI	8 procesos	0.923318	1.56x	0.20
MPI	16 procesos	0.828499	1.74x	0.11
MPI	32 procesos	1.00376	1.44x	0.04
Híbrido	3 procs x 2 hilos	1.03479	1.39x	0.23
Híbrido	3 procs x 4 hilos	0.847219	1.70x	0.14
Híbrido	3 procs x 8 hilos	0.752591	1.92x	0.08
Híbrido	4 procs x 8 hilos	0.812051	1.78x	0.06
Híbrido	8 procs x 4 hilos	0.778828	1.85x	0.06
Híbrido	16 procs x 2 hilos	0.778815	1.85x	0.06

Figura B.5: Resultados del benchmark en entorno cluster para imagen de 2000x2000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	1.510772	1.00x	1.00
OpenMP	2 hilos	1.406297	1.07x	0.54
OpenMP	4 hilos	1.405874	1.07x	0.27
OpenMP	8 hilos	1.406725	1.07x	0.13
OpenMP	16 hilos	1.418437	1.07x	0.07
OpenMP	32 hilos	1.417058	1.07x	0.03
MPI	3 procesos	1.439318	1.05x	0.35
MPI	8 procesos	0.992044	1.52x	0.19
MPI	16 procesos	0.894943	1.69x	0.11
MPI	32 procesos	1.114817	1.36x	0.04
Híbrido	3 procs x 2 hilos	1.100019	1.37x	0.23
Híbrido	3 procs x 4 hilos	0.915643	1.65x	0.14
Híbrido	3 procs x 8 hilos	0.824828	1.83x	0.08
Híbrido	4 procs x 8 hilos	0.857177	1.76x	0.06
Híbrido	8 procs x 4 hilos	0.825466	1.83x	0.06
Híbrido	16 procs x 2 hilos	0.841704	1.79x	0.06

Figura B.6: Resultados del benchmark en entorno cluster para imagen de 2000x2000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	2.243352	1.00x	1.00
OpenMP	2 hilos	2.196568	1.02x	0.51
OpenMP	4 hilos	2.195392	1.02x	0.26
OpenMP	8 hilos	2.196494	1.02x	0.13
OpenMP	16 hilos	2.195356	1.02x	0.06
OpenMP	32 hilos	2.195179	1.02x	0.03
MPI	3 procesos	2.187214	1.03x	0.34
MPI	8 procesos	1.158203	1.94x	0.24
MPI	16 procesos	0.92544	2.42x	0.15
MPI	32 procesos	1.067723	2.10x	0.07
Híbrido	3 procs x 2 hilos	1.419027	1.58x	0.26
Híbrido	3 procs x 4 hilos	1.037532	2.16x	0.18
Híbrido	3 procs x 8 hilos	0.853267	2.63x	0.11
Híbrido	4 procs x 8 hilos	0.855539	2.62x	0.08
Híbrido	8 procs x 4 hilos	0.800778	2.80x	0.09
Híbrido	16 procs x 2 hilos	0.909793	2.47x	0.08

Figura B.7: Resultados del benchmark en entorno cluster para imagen de 2000x2000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	2.253848	1.00x	1.00
OpenMP	2 hilos	2.207473	1.02x	0.51
OpenMP	4 hilos	2.206545	1.02x	0.26
OpenMP	8 hilos	2.285057	0.99x	0.12
OpenMP	16 hilos	2.162358	1.04x	0.07
OpenMP	32 hilos	2.21655	1.02x	0.03
MPI	3 procesos	2.172716	1.04x	0.35
MPI	8 procesos	1.234167	1.83x	0.23
MPI	16 procesos	0.951437	2.37x	0.15
MPI	32 procesos	1.038206	2.17x	0.07
Híbrido	3 procs x 2 hilos	1.425874	1.58x	0.26
Híbrido	3 procs x 4 hilos	1.046614	2.15x	0.18
Híbrido	3 procs x 8 hilos	0.888088	2.54x	0.11
Híbrido	4 procs x 8 hilos	0.889328	2.53x	0.08
Híbrido	8 procs x 4 hilos	0.858714	2.62x	0.08
Híbrido	16 procs x 2 hilos	0.879349	2.56x	0.08

Figura B.8: Resultados del benchmark en entorno cluster para imagen de 2000x2000, filtro 5x5 y 9 rangos de posterización.

Imagen: 5000x5000**Filtro: 3x3, Posterizado: 3 rangos**

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	9.110487	1.00x	1.00
OpenMP	2 hilos	8.181822	1.11x	0.56
OpenMP	4 hilos	8.173508	1.11x	0.28
OpenMP	8 hilos	8.602096	1.06x	0.13
OpenMP	16 hilos	8.874222	1.03x	0.06
OpenMP	32 hilos	8.673202	1.05x	0.03
MPI	3 procesos	8.143475	1.12x	0.37
MPI	8 procesos	5.236314	1.74x	0.22
MPI	16 procesos	4.571317	1.99x	0.12
MPI	32 procesos	5.801641	1.57x	0.05
Híbrido	3 procs x 2 hilos	6.138483	1.48x	0.25
Híbrido	3 procs x 4 hilos	4.972677	1.83x	0.15
Híbrido	3 procs x 8 hilos	4.42153	2.06x	0.09
Híbrido	4 procs x 8 hilos	4.442218	2.05x	0.06
Híbrido	8 procs x 4 hilos	4.233731	2.15x	0.07
Híbrido	16 procs x 2 hilos	4.315193	2.11x	0.07

Figura B.9: Resultados del benchmark en entorno cluster para imagen de 5000x5000, filtro 3x3 y 3 rangos de posterización.

Filtro: 3x3, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	10.223752	1.00x	1.00
OpenMP	2 hilos	9.303469	1.10x	0.55
OpenMP	4 hilos	9.294117	1.10x	0.28
OpenMP	8 hilos	9.626707	1.06x	0.13
OpenMP	16 hilos	9.684011	1.06x	0.07
OpenMP	32 hilos	9.727609	1.05x	0.03
MPI	3 procesos	9.683798	1.06x	0.35
MPI	8 procesos	6.361911	1.61x	0.20
MPI	16 procesos	5.776943	1.77x	0.11
MPI	32 procesos	7.042836	1.45x	0.05
Híbrido	3 procs x 2 hilos	7.296152	1.40x	0.23
Híbrido	3 procs x 4 hilos	6.054184	1.69x	0.14
Híbrido	3 procs x 8 hilos	5.496362	1.86x	0.08
Híbrido	4 procs x 8 hilos	5.56596	1.84x	0.06
Híbrido	8 procs x 4 hilos	5.312799	1.92x	0.06
Híbrido	16 procs x 2 hilos	5.491643	1.86x	0.06

Figura B.10: Resultados del benchmark en entorno cluster para imagen de 5000x5000, filtro 3x3 y 9 rangos de posterización.

Filtro: 5x5, Posterizado: 3 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	13.798248	1.00x	1.00
OpenMP	2 hilos	13.194396	1.05x	0.52
OpenMP	4 hilos	13.206031	1.04x	0.26
OpenMP	8 hilos	13.163794	1.05x	0.13
OpenMP	16 hilos	13.225977	1.04x	0.07
OpenMP	32 hilos	13.18571	1.05x	0.03
MPI	3 procesos	13.212348	1.04x	0.35
MPI	8 procesos	6.513229	2.12x	0.26
MPI	16 procesos	4.926443	2.80x	0.18
MPI	32 procesos	5.740116	2.40x	0.08
Híbrido	3 procs x 2 hilos	8.742038	1.58x	0.26
Híbrido	3 procs x 4 hilos	5.855753	2.36x	0.20
Híbrido	3 procs x 8 hilos	4.702978	2.93x	0.12
Híbrido	4 procs x 8 hilos	4.748632	2.91x	0.09
Híbrido	8 procs x 4 hilos	4.315156	3.20x	0.10
Híbrido	16 procs x 2 hilos	4.433721	3.11x	0.10

Figura B.11: Resultados del benchmark en entorno cluster para imagen de 5000x5000, filtro 5x5 y 3 rangos de posterización.

Filtro: 5x5, Posterizado: 9 rangos

Implementación	Configuración	Tiempo (s)	Speedup	Eficiencia
Secuencial	N/A	14.26659	1.00x	1.00
OpenMP	2 hilos	13.716864	1.04x	0.52
OpenMP	4 hilos	13.680017	1.04x	0.26
OpenMP	8 hilos	13.649838	1.05x	0.13
OpenMP	16 hilos	13.708043	1.04x	0.07
OpenMP	32 hilos	13.240722	1.08x	0.03
MPI	3 procesos	13.635578	1.05x	0.35
MPI	8 procesos	7.022422	2.03x	0.25
MPI	16 procesos	5.418419	2.63x	0.16
MPI	32 procesos	6.153388	2.32x	0.07
Híbrido	3 procs x 2 hilos	9.113853	1.57x	0.26
Híbrido	3 procs x 4 hilos	6.323517	2.26x	0.19
Híbrido	3 procs x 8 hilos	5.127736	2.78x	0.12
Híbrido	4 procs x 8 hilos	5.165777	2.76x	0.09
Híbrido	8 procs x 4 hilos	4.812774	2.96x	0.09
Híbrido	16 procs x 2 hilos	4.841827	2.95x	0.09

Figura B.12: Resultados del benchmark en entorno cluster para imagen de 5000x5000, filtro 5x5 y 9 rangos de posterización.